Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 1

Дисциплина МДК 03.02

Классификация стандартов по категориям и видам

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Классификация документов.....3

1.2 Три ключевых стандарта.....4

1.3 Программа и результат.....4

1.4 Выводы.....5

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6

ПРИЛОЖЕНИЕ А Листинг исходного кода.....8

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [6]. Оформление выполнено по положению института [5].

1.1 Классификация документов

Цель работы — разделить стандарты процессов, качества продукта и документирования. Вид документа и предмет регулирования я рассматриваю отдельно: ГОСТ Р может описывать качество, а международный документ — процессы. Принадлежность к ISO сама по себе не определяет категорию.

ГОСТ 19.201-78 задаёт состав ТЗ, ГОСТ 19.301-79 — содержание программы и методики испытаний, ГОСТ Р 59792-2021 — виды испытаний автоматизированных систем [1] [2] [4].

Сведения представлены в таблице 1.

Таблица 1 — Классификационная матрица

Стандарт	Категория	Применение
ГОСТ 19.201-78	Документирование · ГОСТ	ТЗ и критерии приёмки
ГОСТ 19.301-79	Документирование · ГОСТ	Программа и методика испытаний
ГОСТ Р 59792-2021	Процессы · ГОСТ Р	Виды испытаний АС
ГОСТ Р ИСО/МЭК 25010-2015	Качество продукта · ГОСТ Р	Восемь характеристик профиля
ISO/IEC/IEEE 12207:2026	Процессы · ISO/IEC/IEEE	Жизненный цикл и верификация
ISO/IEC 25010:2023	Качество продукта · ISO/IEC	Новая модель с девятью характеристиками
ISO/IEC/IEEE 29119-3:2021	Документирование · ISO/IEC/IEEE	Тест-кейсы и отчёт
IEEE 1012-2024	Процессы · IEEE	Верификация и валидация

Источник: составлено автором на основе [1], [2], [4], [3], [9], [8], [10], [7]

1.2 Три ключевых стандарта

Первым ключевым документом я выбрал ГОСТ 19.201-78. В ТЗ LS-TZ-01 зафиксированы FR-01–FR-10: права, типы файлов, ограничения и критерии приёмки. Без такого основания нельзя доказать, что отказ загрузки читателю или блокирование ZIP соответствует ожидаемому поведению [1].

Вторым ключевым документом принят ГОСТ Р ИСО/МЭК 25010-2015: по нему построен требуемый заданием профиль из восьми характеристик. Карантин относится к защищённости, а отсутствие частично сохранённого набора — к надёжности. Баллы являются оценкой проекта и не означают сертификацию [3].

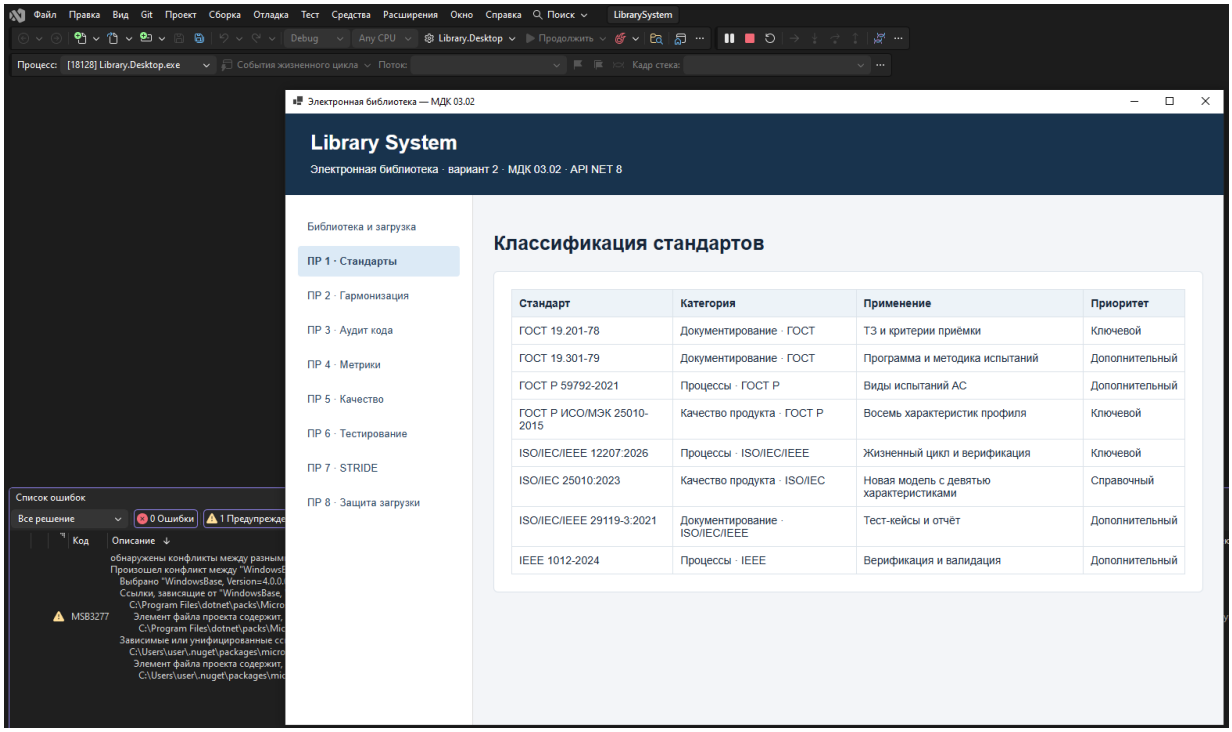
Третий ключевой документ — ISO/IEC/IEEE 12207:2026. Он связывает процессы жизненного цикла. В работе эта связь представлена требованиями, исходным образцом before, исправленным кодом и журналами тестов. Обязательность этих трёх документов установлена принятой базой проекта, а не утверждением об обязательности всех стандартов для любого приложения [9].

Актуальная ISO/IEC 25010:2023 содержит девять характеристик; я учитываю её отдельно и не приписываю ей старую восьмичастную схему. Для состава отчётности использованы ISO/IEC/IEEE 29119-3:2021 и IEEE 1012-2024 [8] [10] [7].

1.3 Программа и результат

Вкладка «ПР 1 · Стандарты» показывает матрицу, составленную для проекта. Входные данные хранятся в artifacts/standards.json. Утилита C# также может сформировать JSON и CSV в verification. Программа отображает подготовленную классификацию стандартов.

В окне программы показана составленная матрица стандартов (рисунок 1).



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Классификация стандартов в приложении, запущенном из Visual Studio

1.4 Выводы

Я распределил восемь документов по трём категориям и обосновал ключевую нормативную базу. Проверка редакций позволила согласовать требуемый профиль качества с действующей национальной моделью и отдельно отметить современную международную редакцию.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению. — URL: https://protect.gost.ru/gost/details/b64061cf-6bb2-4c75-aee9-18bb5e26e554 (дата обращения: 08.11.2026) [1].

2. ГОСТ 19.301-79. ЕСПД. Программа и методика испытаний. Требования к содержанию и оформлению. — URL: https://protect.gost.ru/gost/details/b25e0645-091a-425b-8558-3e0c3ce37315 (дата обращения: 08.11.2026) [2].

3. ГОСТ Р ИСО/МЭК 25010-2015. Системная и программная инженерия. Модели качества систем и программных продуктов. — URL: https://protect.gost.ru/gost/details/e5744d86-5a17-4909-96da-b38691326f0c (дата обращения: 08.11.2026) [3].

4. ГОСТ Р 59792-2021. Информационные технологии. Виды испытаний автоматизированных систем. — URL: https://protect.gost.ru/gost/details/60279c95-21dc-430c-bf2c-f626c588576e (дата обращения: 08.11.2026) [4].

5. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [5].

6. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [6].

7. IEEE 1012-2024. Standard for System, Software, and Hardware Verification and Validation. — URL: https://standards.ieee.org/ieee/1012/7324/ (дата обращения: 08.11.2026) [7].

8. ISO/IEC 25010:2023. Systems and software engineering. SQuaRE. Product quality model. — URL: https://www.iso.org/standard/78176.html (дата обращения: 08.11.2026) [8].

9. ISO/IEC/IEEE 12207:2026. Systems and software engineering. Software life cycle processes. — URL: https://www.iso.org/standard/90219.html (дата обращения: 08.11.2026) [9].

10. ISO/IEC/IEEE 29119-3:2021. Software testing. Part 3. Test documentation. — URL: https://www.iso.org/standard/79429.html (дата обращения: 08.11.2026) [10].

ПРИЛОЖЕНИЕ A
Листинг исходного кода

src/Library.Tools/Program.cs

```
using System.Diagnostics;
using System.Text;
using System.Text.Json;
using System.Text.Encodings.Web;
using System.Xml.Linq;
using Library.Tools;
using Library.Core;
Console.OutputEncoding = Encoding.UTF8;
var directory = new DirectoryInfo(Directory.GetCurrentDirectory());
while (directory is not null && !Directory.Exists(Path.Combine(directory.FullName, "artifacts"))) directory =
directory.Parent;
string root = directory?.FullName ?? throw new IOException("Запуск из корня Project");
string verification = Path.Combine(root, "verification");Directory.CreateDirectory(verification);
var options = new JsonSerializerOptions(JsonSerializerDefaults.Web) { WriteIndented = true, Encoder =
JavaScriptEncoder.UnsafeRelaxedJsonEscaping };
void Save(string name, object data) => File.WriteAllText(Path.Combine(verification, name + ".json"),
JsonSerializer.Serialize(data, options));
string command = args.FirstOrDefault() ?? "all";
try
{
  if (command is "all" or "samples")
  {
    Directory.CreateDirectory(Path.Combine(root, "samples"));
    File.WriteAllBytes(Path.Combine(root, "samples/sample.pdf"), Samples.Pdf());
    File.WriteAllBytes(Path.Combine(root, "samples/sample.epub"), Samples.Epub());
    File.WriteAllBytes(Path.Combine(root, "samples/zip-slip.zip"), Samples.Zip(("../escape.pdf",
Samples.Pdf())));
  }
  foreach (string name in new[] { "standards", "harmonization", "quality", "threats", "bugs" })
    if (command == "all" || command == name)
    {
      string text = File.ReadAllText(Path.Combine(root, "artifacts", name + ".json"));
      File.WriteAllText(Path.Combine(verification, name + ".json"), text);
      var rows = JsonDocument.Parse(text).RootElement;
      string Escape(string s) => "\"" + s.Replace("\"", "\"\"") + "\"";
      var lines = new List<string> { string.Join(';', rows[0].EnumerateObject().Select(x => Escape(x.Name))) };
      lines.AddRange(rows.EnumerateArray().Select(row => string.Join(';', row.EnumerateObject().Select(x
=> Escape(x.Value.ToString())))));
      File.WriteAllLines(Path.Combine(verification, name + ".csv"), lines, new UTF8Encoding(true));
      Console.WriteLine(name + ": строк " + rows.GetArrayLength());
    }
  if (command is "all" or "audit" or "metrics")
  {
    string before = File.ReadAllText(Path.Combine(root, "before/LegacyUploadController.cs"));
    string after = File.ReadAllText(Path.Combine(root, "src/Library.Api/UploadController.cs"));
    string catalog = File.ReadAllText(Path.Combine(root, "src/Library.Api/BooksController.cs"));
    var result = new { before = CodeQuality.Measure(before, "LegacyUploadController"),
      after = CodeQuality.Measure(after, "UploadController"), catalog = CodeQuality.Measure(catalog,
"BooksController"),
      beforeAudit = CodeQuality.Audit(before), afterAudit = CodeQuality.Audit(after), catalogAudit =
CodeQuality.Audit(catalog) };
    Save("analysis", result);Console.WriteLine(JsonSerializer.Serialize(result, options));
  }
```

```
  if (command == "parse-tests")
  {
    string path = args.Length > 1 ? args[1] : Path.Combine(verification, "library-tests.trx");
    var tests = XDocument.Load(path).Descendants().Where(x => x.Name.LocalName == "UnitTestResult")
      .Select(x => new { name = (string?)x.Attribute("testName"), outcome = (string?)x.Attribute("outcome"),
duration = (string?)x.Attribute("duration") }).OrderBy(x => x.name).ToArray();
    if (tests.Length == 0) throw new IOException("Нет результатов TRX");
    var result = new { total = tests.Length, passed = tests.Count(x => x.outcome == "Passed"), failed =
tests.Count(x => x.outcome == "Failed"), skipped = tests.Count(x => x.outcome is not ("Passed" or "Failed")),
tests };
    Save("tests", result);Console.WriteLine($"Всего {result.total}; Passed {result.passed}; Failed
{result.failed}; Skipped {result.skipped}");
    if (result.failed > 0 || result.skipped > 0) Environment.ExitCode = 1;
  }
  if (command == "api-checks")
  {
    using var client = new HttpClient { BaseAddress = new Uri(args.Length > 1 ? args[1] :
"http://127.0.0.1:5352") };
    using var response = await client.PostAsync("/lab/scenarios", null);response.EnsureSuccessStatusCode();
    string text = await response.Content.ReadAsStringAsync();File.WriteAllText(Path.Combine(verification,
"api-scenarios.json"), text);Console.WriteLine(text);
    if (JsonDocument.Parse(text).RootElement.EnumerateArray().Any(x => !
x.GetProperty("passed").GetBoolean())) Environment.ExitCode = 1;
  }
  if (command == "open-ui")
  {
    var url = new Uri(args[1]);using var client = new HttpClient { Timeout = TimeSpan.FromSeconds(2) };bool
ready = false;
    for (int i = 0; i < 30 && !ready; i++)
    {
      try { using var response = await client.GetAsync(new Uri(url, "/health"));ready =
response.IsSuccessStatusCode; }
      catch (HttpRequestException) { } catch (TaskCanceledException) { }
      if (!ready) await Task.Delay(500);
    }
    if (!ready) throw new IOException("Сервер ещё не запущен");
    Process.Start(new ProcessStartInfo(url.AbsoluteUri) { UseShellExecute = true });
  }
}
catch (Exception e) { Console.Error.WriteLine("Операция не выполнена: " +
e.Message);Environment.ExitCode = 1; }
```